[10]

Merge Conflicts

In Chapter 9, you carried out a three-way merge in which you did not experience any merge conflicts.
In this chapter, you're going to learn about merge conflicts, how they arise, and how to resolve them by walking through a hands-on example of a three-way merge with a merge conflict.

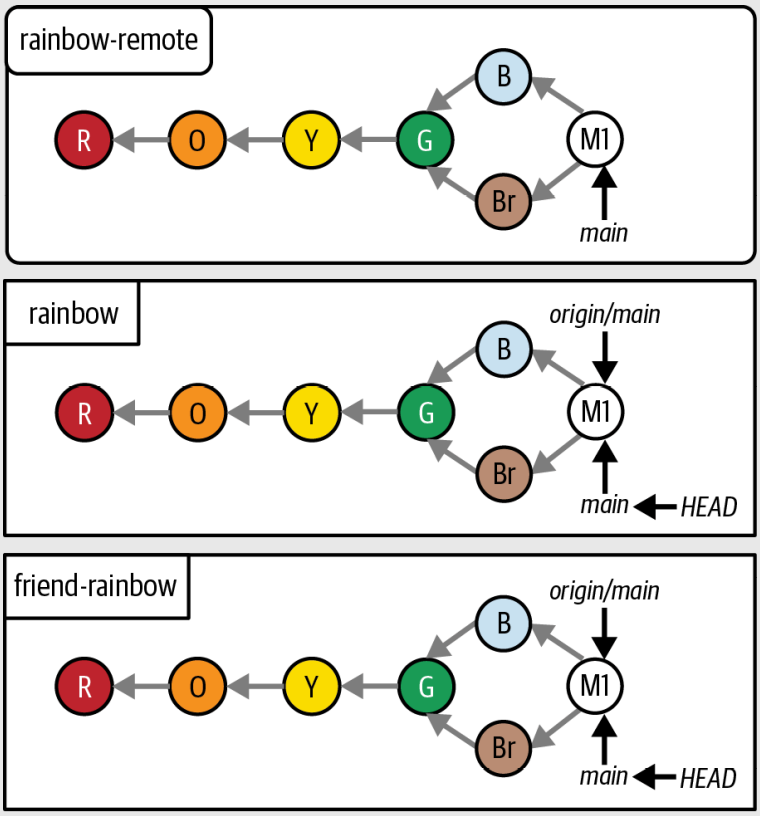
State of the Local and Remote Repositories

At the start of this chapter, you should have two local repositories called rainbow and friend-rainbow and one remote repository called rainbow-remote. All three of these repositories should be in sync, with the same commits and branches. As usual, I recommend that you use two separate text editor windows and command line windows for working with the rainbow repository and the friend-rainbow repository.

Visualize it 10-1 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 through Chapter 9.

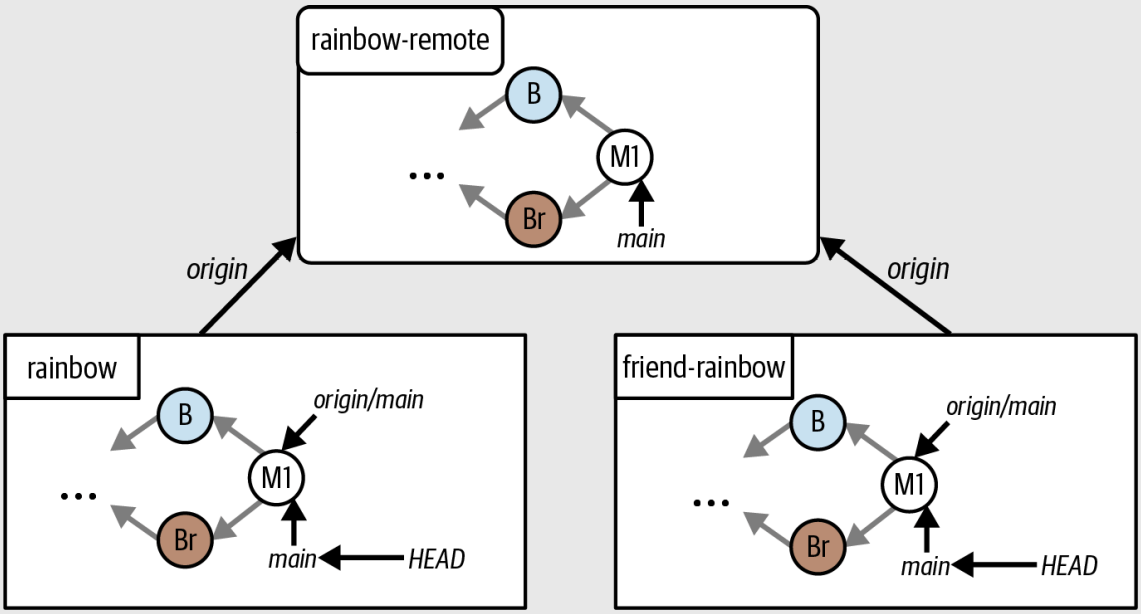
To focus on the commits you are going to make in this chapter, from here on I'll simplify the Visualize It diagrams and show only the last three commits that are part of the main branch in all the repositories: the blue commit, the brown commit, and the M1 merge commit. This representation is shown in Visualize it 10-2.

[VISUALIZE IT 10-1]



The Rainbow project at the start of Chapter 10 with all the commits since Chapter 1

[VISUALIZE IT 10-2]



A simplified representation of the Rainbow project at the start of Chapter 10, showing just the last three commits on the main branch in all the repositories

Introducing Merge Conflicts

In Chapter 9, we covered what happens when you work on a project at the same time as someone else but you each make changes to different files: you edited the othercolors.txt file, while your friend edited the rainbowcolors.txt file. This meant that you were able to do a three-way merge without any merge conflicts.

Now, we will cover what happens when you want to integrate work where there will be a merge conflict. As you learned in the previous chapter, merge conflicts arise when you merge two branches where different changes have been made to the same parts in the same file(s), or if in one branch a file was deleted that was edited in the other branch. In these cases Git is unable to automatically merge the files, and therefore you must do it manually.

Merge conflicts can arise during the process of merging as well as the process of rebasing. In this chapter we will cover an example of a merge conflict in the Rainbow project while merging, and in Chapter 11 we will cover an example of a merge conflict in the Rainbow project while rebasing.

Bear in mind that it is normal for merge conflicts to arise, and if they do, it does not mean that someone did something wrong while working on a project. Resolving merge conflicts is a regular part of working on Git projects. Let's take a look at Example Book Project 10-1 to see an example of how such a situation might occur.

EXAMPLE BOOK PROJECT 10-1

Let's imagine a couple of scenarios where I may experience merge conflicts in my Book project. As you know, the project consists of 10 text files, one for each chapter of the book, called chapter_one.txt, chapter_two.txt, and so on. My primary line of development is the main branch, and I've decided together with my editor and my coauthor that whenever I work on a chapter I will make a branch off the main branch; then, after they have both approved the work I have done on that branch, I can merge the branch back into the main branch. I've also agreed with my coauthor that only one of us should work on any one chapter at a time, and this way we can often avoid merge conflicts.

But now, let's imagine what happens if my coauthor and I accidentally miscommunicate. We both decide to edit chapter 3 at the same time, and we both make branches off the main branch to work on this chapter at the same time. I make the chapter_three branch, and my coauthor makes the chapter_three_coauthor branch. In this case, we are both going to edit the same file, chapter_three.txt.

Now, let's assume that my coauthor merges their work on the chapter_three_coauthor branch into the main branch first. When I go to merge my chapter_three branch into the most up-to-date version of the main branch, I will find that not only do I have to carry out a three-way merge, but I will also have to resolve merge conflicts.

This is because the version of the chapter_three.txt file in my chapter_three branch and the version of the file in the main branch have both been edited in different ways since I made my branch off the main branch. Git isn't able to automatically merge the work. It needs me to go in and decide exactly what the final version of the chapter_three.txt file that will be included in the merge commit should look like.

This is one scenario in which I may experience merge conflicts. It occurs when the same file has been edited in different ways in the two branches involved in the merge.

Next, let's explore another scenario in which a merge conflict might arise. Let's imagine that my coauthor and I both agree that the last chapter in the book, chapter 10, isn't any good. However, we again miscommunicate about what we should do about it.

We accidentally both make branches off the main branch at the same time: I make the chapter_ten branch and my coauthor makes the chapter_ten_coauthor branch. In my chapter_ten branch, I decide to delete the chapter_ten.txt file because I think we should simply remove that chapter from our book. However, my coauthor, working on their chapter_ten_coauthor branch, instead decides to edit the chapter_ten.txt file to make it better.

Again, my coauthor merges their branch into the main branch first. And when I go to merge my branch into the most up-to-date main branch through a three-way merge, I will again encounter a merge conflict. This is another scenario where Git isn't able to automatically merge the work; it can't determine whether it should delete the chapter_ten.txt file or retain the edited version. To recap, this situation occurs when I am integrating two branches where in one branch a specific file has been edited and in the other branch that same file has been deleted.

Example Book Project 10-1 described a couple of scenarios where merge conflicts might arise. Now let's take a look at how you can resolve them.

How to Resolve Merge Conflicts

When merge conflicts happen, you will see a set of special markers in each of the files involved that indicate where the conflicts occur. These markers, called *conflict markers*, consist of seven left angle brackets (<<<<<<<), seven equals signs (=======), and seven right angle brackets (>>>>>>>), as well as references to the branches involved in the merge. Figure 10-1 shows an example. Above the row of equals signs you will see the content of the target branch, and below it you will see the content of the source branch.

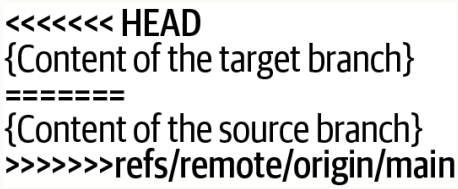


FIGURE 10-1

An example of the conflict markers that appear when there is a merge conflict

There are two steps to resolving merge conflicts:

1. Decide what to keep, edit the content, and remove the conflict markers.
2. Add the file(s) you have edited to the staging area and commit your changes.

We will cover each of these steps in detail when you practice resolving merge conflicts in the Rainbow project later in this chapter. To prepare for that, you need to create a situation with divergent development histories in your local repositories. You'll do that next.

Setting Up a Merge Conflict Scenario

To set up a three-way merge with merge conflicts, you and your friend have to make different changes to the same part in the same file. First, go to Follow Along 10-1 to add the color indigo to the list of colors in the rainbowcolors.txt file in your rainbow repository and push the updated main branch to the remote repository.

[FOLLOW ALONG 10-1]

1. In the rainbow project directory in your text editor, in the rainbowcolors.txt file, add “Indigo is the sixth color of the rainbow.” on line 6. Then save the file.

2. rainbow $ **git add rainbowcolors.txt**

3. rainbow $ **git commit -m "indigo"**
   [main 9b0a614] indigo
   1 file changed, 2 insertions(+), 1 deletion(-)

4. rainbow $ **git push**
   Enumerating objects: 5, done.
   Counting objects: 100% (5/5), done.
   Delta compression using up to 4 threads
   Compressing objects: 100% (3/3), done.
   Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
   Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
   remote: Resolving deltas: 100% (1/1), completed with 1 local object.
   To github.com:gitlearningjourney/rainbow-remote.git
   2258399..9b0a614 main -> main

5. rainbow $ **git log**
   commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (**HEAD -> main**, **origin/main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 08:36:11 2022 +0100
   indigo
   commit 225839938563c7458af81daca7beb782dfcbfb27
   Merge: 342bbfc 7f0a87a
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 13:05:46 2022 +0100
   Merge remote-tracking branch 'refs/remotes/origin/main'

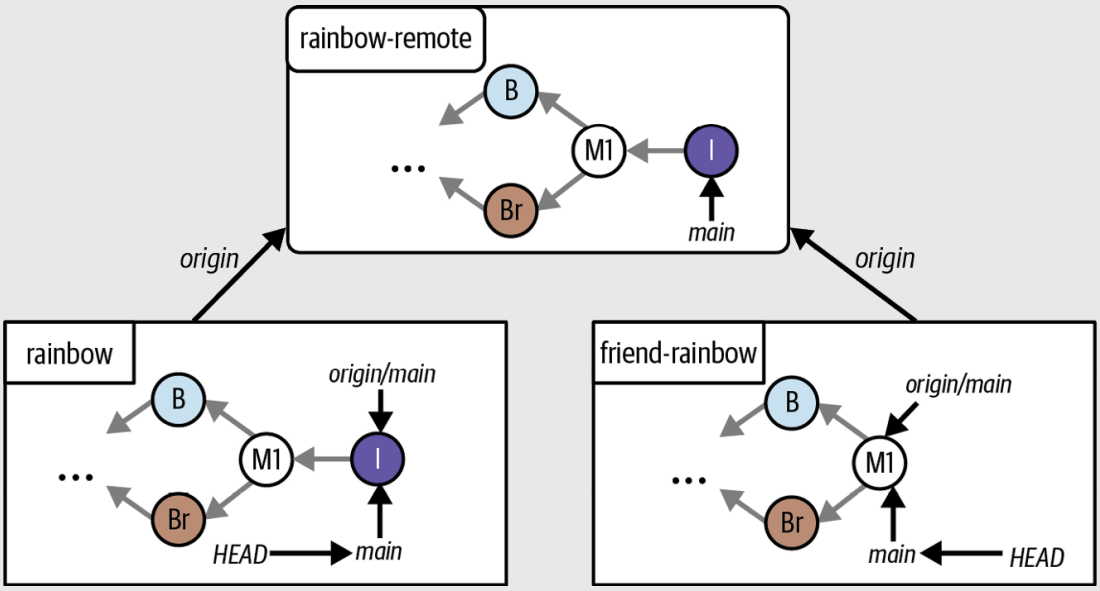
6. Go to the rainbow-remote repository on your hosting service and refresh the page. You should see the indigo commit there.

What to notice:

- In the rainbow repository and in the rainbow-remote repository, there is an indigo commit.

These changes are illustrated in Visualize it 10-3.

[VISUALIZE IT 10-3]



The Rainbow project after you make the indigo commit in your rainbow repository and push the updated main branch to the remote repository

Now, to create a situation where you will have a merge conflict, in Follow Along 10-2 your friend will add the color violet to the same line in the rainbowcolors.txt file and make a commit without first pulling the changes you made from the rainbow-remote repository into their local main branch.

[FOLLOW ALONG 10-2]

1. In the the rainbowcolors.txt file in the friend-rainbow project directory in your text editor, add “Violet is the seventh color of the rainbow.” on line 6. Save the file.

2. friend-rainbow $ **git add rainbowcolors.txt**

3. friend-rainbow $ **git commit -m "violet"**
   [main 6ad5c15] violet
   1 file changed, 2 insertions(+), 1 deletion(-)

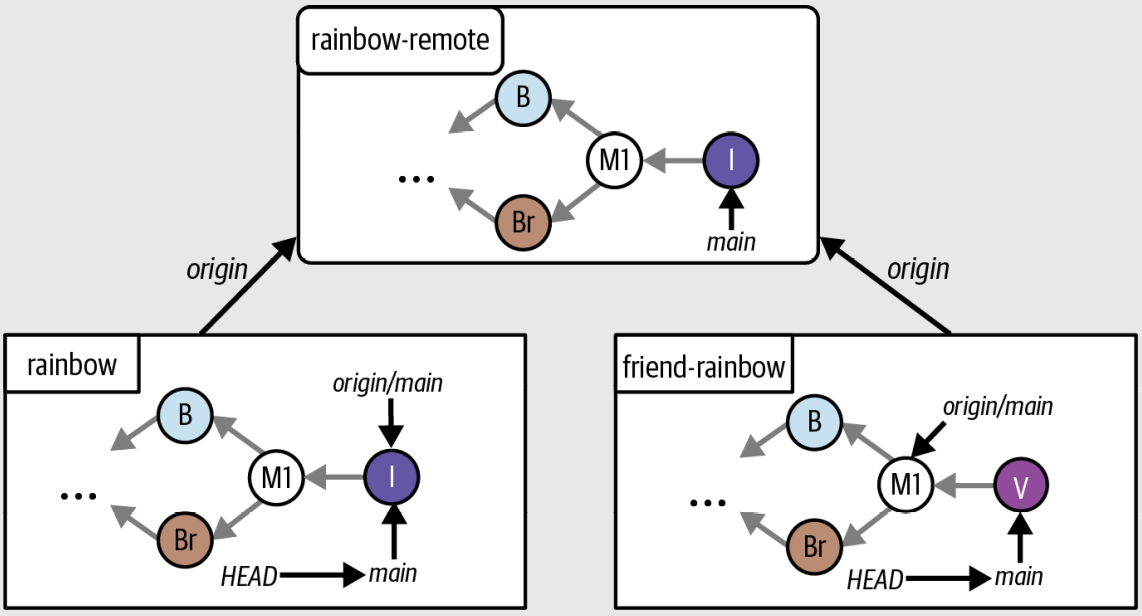
4. friend-rainbow $ **git log**
   commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 08:41:25 2022 +0100
   violet
   commit 225839938563c7458af81daca7beb782dfcbfb27 (**origin/main**, **origin/HEAD**)
   Merge: 342bbfc 7f0a87a
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 13:05:46 2022 +0100
   Merge remote-tracking branch 'refs/remotes/origin/main'

What to notice:

- Your friend made the violet commit.

This is illustrated in Visualize it 10-4.

[VISUALIZE IT 10-4]



The Rainbow project after your friend makes the violet commit in the friend-rainbow repository without first pulling the latest work on the main branch in the remote repository

What to notice:

- In the friend-rainbow repository, the local main branch points to the violet commit.
- In the rainbow repository, the local main branch points to the indigo commit.

At this point, your friend will need to fetch your changes from the remote repository and integrate them before they can push their changes to the remote repository. As you learned in "Pushing to the Remote Repository" on page 129, they can use the git status command after fetching your changes to check whether their local main branch has diverged from the remote main branch. Go to Follow Along 10-3 to see this in action.

[FOLLOW ALONG 10-3]

1. friend-rainbow $ **git fetch**

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), 322 bytes | 53.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
c5941f8..6f2ea44 main -> origin/main
```

2. friend-rainbow $ **git status**

```
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)
nothing to commit, working tree clean
```

3. friend-rainbow $ **git log --all**

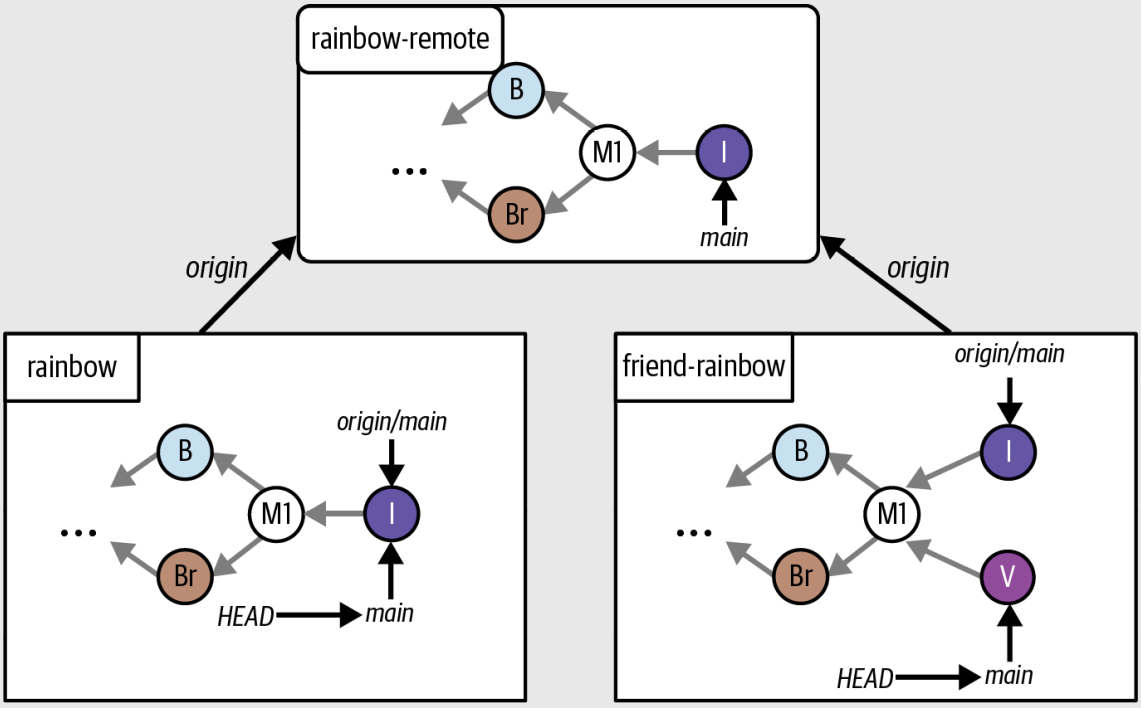
```
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (origin/main,
origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:36:11 2022 +0100
indigo
```

What to notice:

- In step 2, the output of the git status command states that Your branch and 'origin/main' have diverged, and have 1 and 1 different commits each, respectively.
- In the friend-rainbow repository, the origin/main remote-tracking branch points to the indigo commit; however, the remote changes have not yet been merged into the local main branch.

These observations are shown in Visualize it 10-5.

[VISUALIZE IT 10-5]



The Rainbow project after your friend fetches the changes from the remote repository to the friend-rainbow repository

Next, your friend is going to carry out a three-way merge to integrate the latest changes on the remote main branch into their local main branch so they can push the updated main branch to the remote repository. Before they perform the merge and resolve the merge conflicts in the friend-rainbow repository, let's take a closer look at what that process entails.

The Merge Conflict Resolution Process

As mentioned earlier in this chapter, there are two steps to resolving merge conflicts. In this section we'll walk through them in more detail. As a reminder, the first step is to decide what to keep, edit the content, and remove the conflict markers, and the second is to add all the changes to the staging area and commit. Let's go over each of them in turn.

STEP 1

After you execute the git merge command in a situation where a merge conflict arises, Git will identify the conflict and will insert conflict markers indicating the location(s) of the conflicting content. The first step in resolving merge conflicts is to decide what to keep, edit the content in the file(s) where the conflicts occur, and remove the conflict markers. Once you're done making the necessary changes, you must also make sure to save the file or files in your text editor. In the Rainbow project there will be a merge conflict in only one file, but in other Git projects you work on there may be merge conflicts in multiple files.

To resolve the merge conflict in the Rainbow project, your friend will keep both the sentence about the indigo color and the sentence about the violet color. They want to make sure they're in the correct order, so they will put the sentence about indigo first (which will end up being on line 6) and the sentence about violet second (this will end up on line 7). See Figure 10-2 for an example of what the rainbowcolors.txt file will look like before and after your friend makes this change.

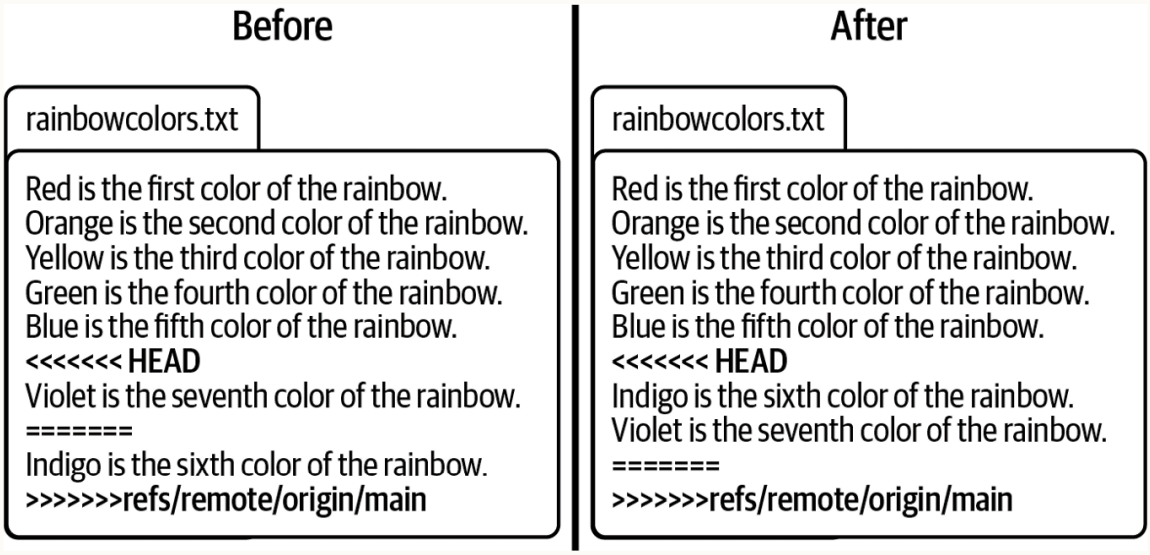


FIGURE 10-2

The rainbowcolors.txt file before and after your friend decides what to keep and edits the content

Next, your friend will need to remove the conflict markers from the rainbowcolors.txt file, as seen in Figure 10-3, and save the file.

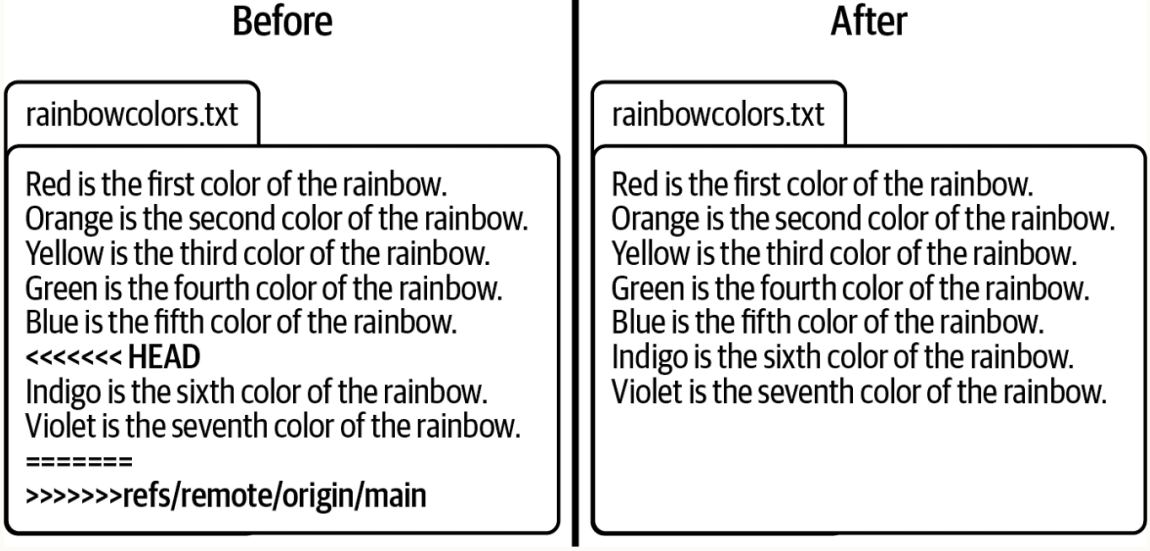


FIGURE 10-3

The rainbowcolors.txt file before and after your friend removes the conflict markers

STEP 2

After your friend has finished editing the content, they're ready for the second step: adding the updated file(s) to the staging area and making a commit. In the Rainbow project there is only one file with a merge conflict, so they'll need to add only one file to the staging area. However, as mentioned previously, it's common to have merge conflicts in multiple files, in which case you have to add all the modified files to the staging area.

In Figure 10-4, you can see an example of what the commit history in the friend-rainbow repository will look like before and after your friend completes the merge. This merge will result in the M2 merge commit, because this is a three-way merge.

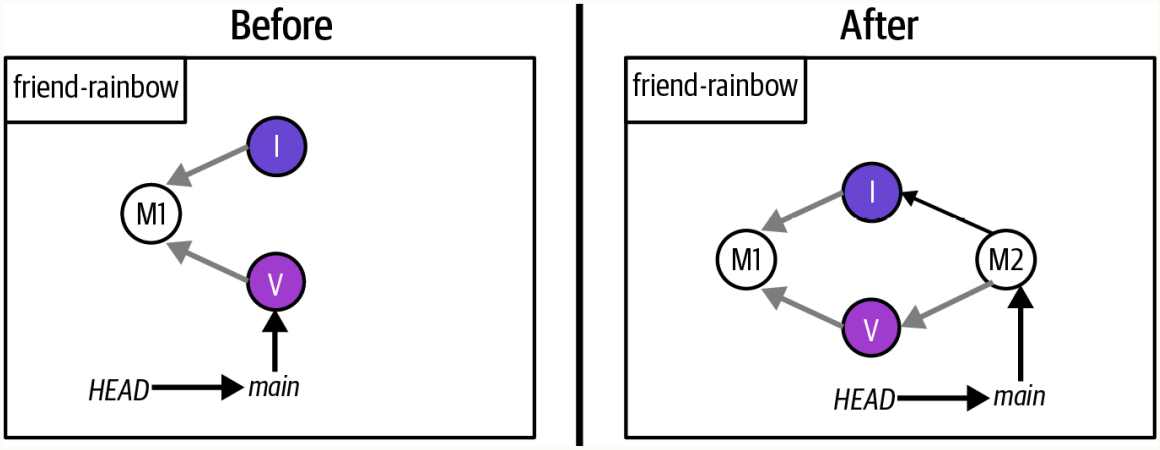


FIGURE 10-4

The commit history before and after your friend completes the three-way merge

Now that you're familiar with the two steps involved in resolving merge conflicts, let's briefly go over what to do if you decide you don't want to continue with a merge.

ABORTING A MERGE

If at any point during a merge with conflicts you decide that you don't want to continue integrating two branches, you can choose to stop or "abort" the merge by using the git merge command with the --abort option. This will return all your files to the state they were in before the merge.

[SAVE THE COMMAND]

git merge --abort

Stop the merge process and go back to the state before the merge

Now that you know how to resolve merge conflicts and what to do if you ever want to abort a merge and the conflict resolution process, let's see an example of resolving merge conflicts in practice.

Resolving Merge Conflicts in Practice

In Follow Along 10-4, your friend is going to carry out the three-way merge and resolve the merge conflicts. Throughout the merge operation, they will use the git status command to see information about the conflict resolution process.

[FOLLOW ALONG 10-4]

1. friend-rainbow $ **git merge origin/main**
   Auto-merging rainbowcolors.txt
   CONFLICT (content): Merge conflict in rainbowcolors.txt
   Automatic merge failed; fix conflicts and then commit the result.

2. friend-rainbow $ **git status**
   On branch main
   Your branch and 'origin/main' have diverged,
   and have 1 and 1 different commits each, respectively.
   (use "git pull" to merge the remote branch into yours)
   You have unmerged paths.
   (fix conflicts and run "git commit")
   (use "git merge --abort" to abort the merge)
   Unmerged paths:
   (use "git add <file>..." to mark resolution)
   both modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

3. Take a deep breath, go to the friend-rainbow project directory in your text editor, and look at the rainbowcolors.txt file. Find the conflict markers and the conflicting content.

4. Carry out step 1 of resolving merge conflicts, which is to choose what to keep, edit the content, and remove the conflict markers. In your case, you are going to keep all of the content; that is, the changes from violet commit and the changes from the indigo commit. The comment about the color indigo will go on line 6, and the comment about the color violet will go on line 7. Make sure to save the rainbowcolors.txt file when you're done making your edits.

5. friend-rainbow $ **git add rainbowcolors.txt**

6. friend-rainbow $ **git status**
   On branch main
   Your branch and 'origin/main' have diverged,
   and have 1 and 1 different commits each, respectively.
   (use "git pull" to merge the remote branch into yours)

[FOLLOW ALONG 10-4]

All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)
Changes to be committed:
modified: rainbowcolors.txt

7 friend-rainbow $ **git commit -m "merge commit 2"**
[main f10f972] merge commit 2

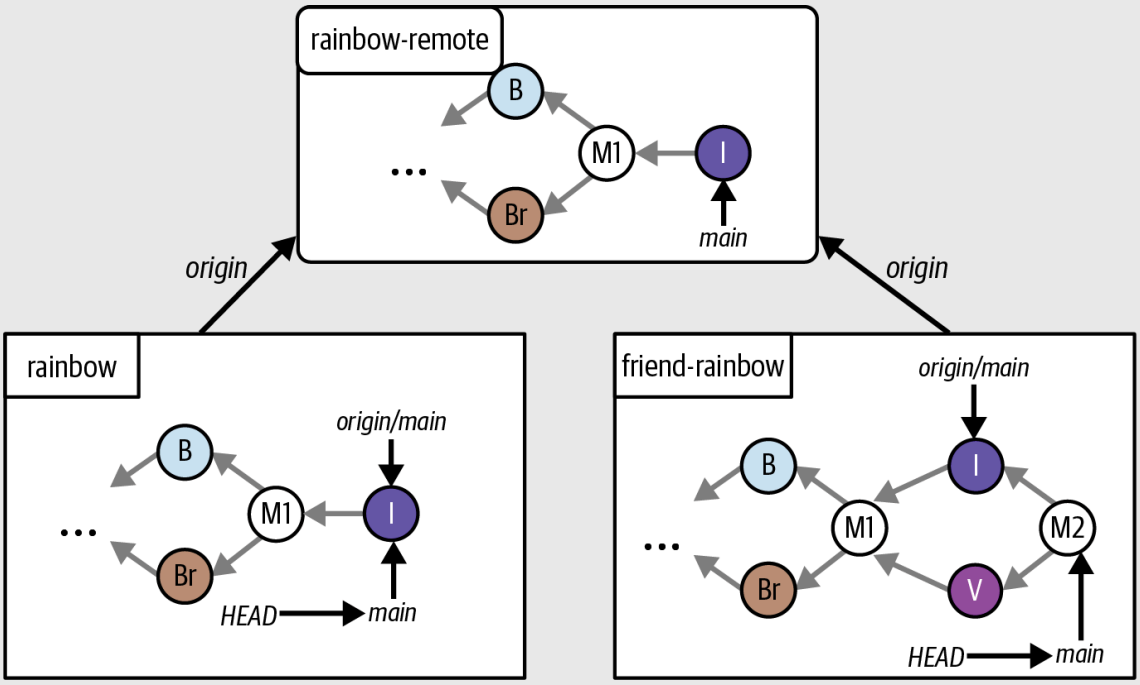
8 friend-rainbow $ **git log**
commit f10f9725e3319af840a3d891ca8950436a219eb0 (**HEAD -> main**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (**origin/main, origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:36:11 2022 +0100
indigo

What to notice:

- In step 8, the git log output shows:
  - The local main branch points to merge commit 2.
  - The two parent commits of merge commit 2 are 6ad5c15 (the violet commit) and 9b0a614 (the indigo commit). Recall that the commit hashes in your friend-rainbow repository will be different from the ones in this book because commit hashes are unique.

These observations are illustrated in Visualize it 10-6, where the new merge commit is represented as M2.

[VISUALIZE IT 10-6]



The Rainbow project after your friend resolves the merge conflicts in the friend-rainbow repository and makes another merge commit

Your friend successfully carried out the three-way merge and resolved the merge conflicts. Next, let's discuss the importance of staying up to date with a remote repository.

Staying Up to Date with a Remote Repository

It is more time-consuming to integrate changes from one branch into another when there are merge conflicts. In the Rainbow project example, there was only one file with a small merge conflict. However, in real-world projects there may be many more files with way more complicated merge conflicts to resolve. Staying up to date with changes made on relevant branches in a remote repository is important because it can help limit the number of potential merge conflicts you may have to resolve later down the road.

Whenever you create a new branch, it is recommended that you base it off the most up-to-date version of the relevant remote branch in the remote repository. Often this will be the main branch (or whichever branch your team uses as the primary line of development). However, depending on your team's Git workflow, it may be another branch.

If you're working on an existing branch on your own, it is recommended to update your branch with the changes made to the relevant remote branch in the remote repository by merging. And if you find yourself in a situation where you're working on the same branch as someone else, then you must always make sure to fetch and merge any changes that have been made on that branch in the remote repository before continuing to work on it.

With this reminder about staying up to date in mind, let's now make sure that your friend pushes the work they've done on their local main branch to the remote repository and that you sync your local main branch in the rainbow repository with the remote main branch.

Syncing the Repositories

For all the repositories to be in sync, your friend will need to push the new commits on their local main branch to the remote repository, and you will need to pull down all the changes into your local main branch in the rainbow repository. Go to Follow Along 10-5 to complete these steps now.

[FOLLOW ALONG 10-5]

1. friend-rainbow $ **git push**

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 614 bytes | 614.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
9b0a614..f10f972 main -> main
```

2. friend-rainbow $ **git log**

```
commit f10f9725e3319af840a3d891ca8950436a219eb0 (HEAD -> main,
origin/main, origin/HEAD)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
```

3. Go to the command line window where you're in the rainbow repository to execute the following command.

4. rainbow $ **git pull**

```
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 2), reused 6 (delta 2), pack-reused 0
Unpacking objects: 100% (6/6), 594 bytes | 99.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
9b0a614..f10f972 main -> origin/main
Updating 9b0a614..f10f972
Fast-forward
rainbowcolors.txt | 5 ++++-
```

[FOLLOW ALONG 10-5]

1 file changed, 4 insertions(+), 1 deletion(-)

5 rainbow $ **git log**

commit f10f9725e3319af840a3d891ca8950436a219eb0 (**HEAD -> main**, **origin/main**)

Merge: 6ad5c15 9b0a614

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 09:11:06 2022 +0100

merge commit 2

commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e

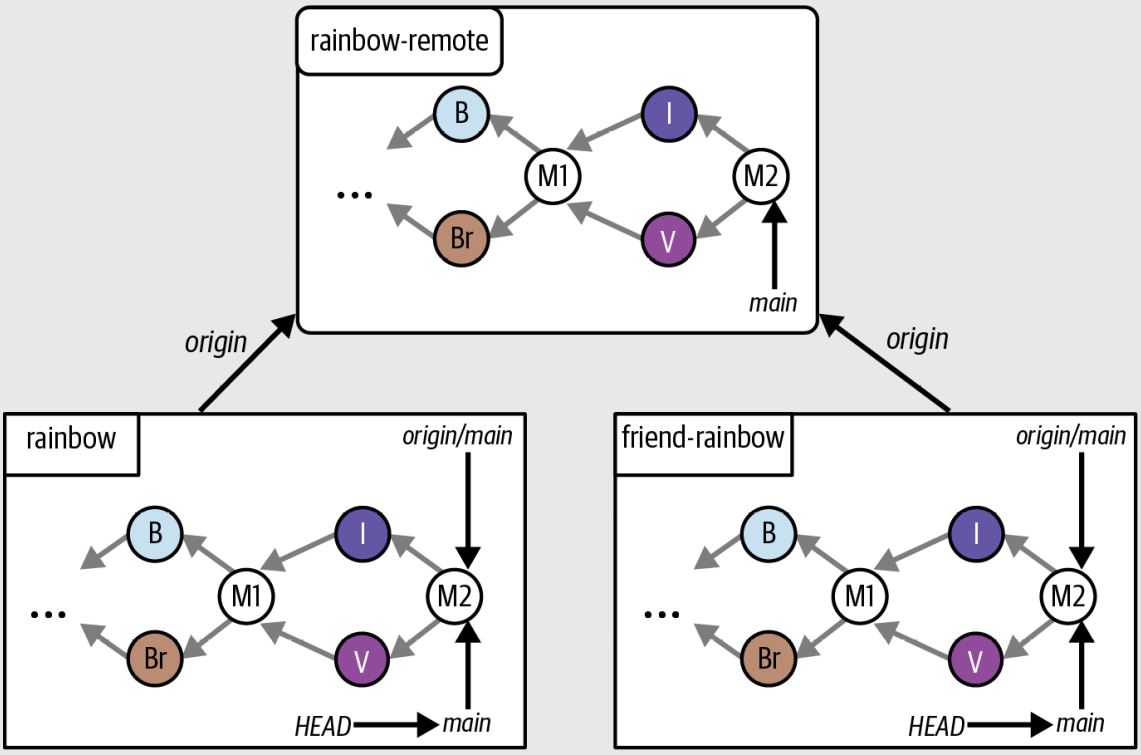
Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 08:41:25 2022 +0100

violet

All three repositories in the Rainbow project are now in sync, as illustrated in Visualize it 10-7.

[VISUALIZE IT 10-7]



The Rainbow project after your friend pushes their changes to the remote repository and you pull their changes down to your rainbow repository

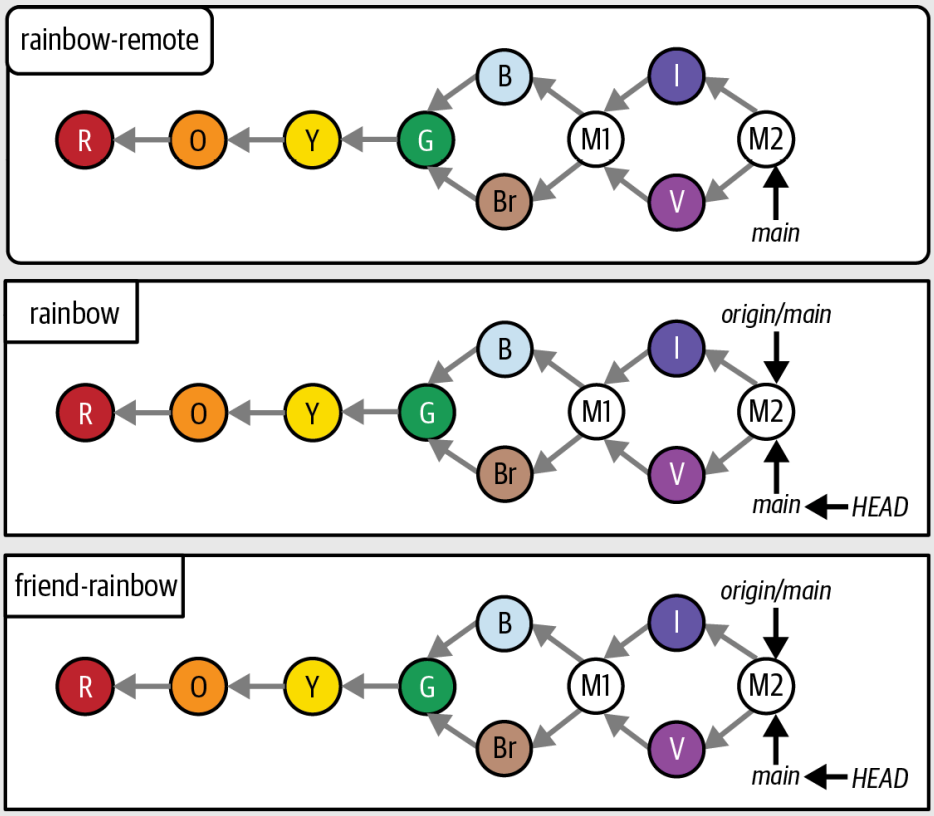
What to notice:

- In the rainbow-remote repository the remote main branch points to the M2 merge commit.
- In the rainbow repository, the local main branch and the origin/main remote-tracking branch point to the M2 merge commit.
- In the friend-rainbow repository, the origin/main remote-tracking branch has updated to point to the M2 merge commit.

State of the Local and Remote Repositories

Visualize it 10-8 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 all the way to the end of this chapter.

[VISUALIZE IT 10-8]



The Rainbow project at the end of Chapter 10 with all the commits since Chapter 1

Summary

In this chapter you learned about merge conflicts, which arise when you merge two branches where different changes have been made to the same parts in the same file(s), or if in one branch a file was deleted that was edited in the other branch. You also learned about the two steps involved in resolving them: deciding what content to keep, removing the conflict markers, and saving the files, then adding the edited files to the staging area and making a commit.

Merge conflicts can arise either during a three-way merge or during the process of rebasing. Up until now we have focused on merging; in Chapter 11 you will learn about rebasing and you will work through a hands-on example in the Rainbow project.

Previous chapter

< Chapter 9. Three-Way Merges

Next chapter

Chapter 11. Rebasing >